



Document de Sécurité – Authentification & Autorisation

1. Objectif du document

Ce document décrit l'architecture de **sécurité** de l'application de gestion des flux financiers. Il définit les mécanismes d'authentification, d'autorisation, de gestion des identités et de protection des données.

L'objectif est de garantir : - La confidentialité des données financières - L'intégrité des opérations - La traçabilité des actions utilisateurs - Une expérience utilisateur moderne et sécurisée

2. Choix technologiques

2.1 Identity Provider (IdP)

- **Keycloak** comme serveur d'identité centralisé
- Support des standards :
 - OAuth 2.0
 - OpenID Connect (OIDC)

2.2 Méthodes d'authentification supportées

Les utilisateurs peuvent s'authentifier via : 1. **Google (Gmail)** – Social Login OAuth2 2. **Facebook** – Social Login OAuth2 3. **Inscription locale** (email + mot de passe) gérée par Keycloak

Keycloak agit comme **fédération d'identité** et point d'entrée unique pour toutes les méthodes.

3. Architecture de sécurité globale

3.1 Vue d'ensemble

- Le frontend Angular n'authentifie jamais directement l'utilisateur
- Toute authentification passe par Keycloak
- Le backend Spring Boot est un **Resource Server** OAuth2

Flux simplifié : 1. L'utilisateur accède à l'application Angular 2. Redirection vers Keycloak si non authentifié 3. Choix du mode d'authentification (Google / Facebook /

Local) 4. Keycloak délivre un **Access Token (JWT)** 5. Angular consomme les APIs Spring Boot avec le token

4. Gestion des utilisateurs

4.1 Création des comptes

- Comptes locaux créés dans Keycloak
- Comptes Google/Facebook liés automatiquement à un utilisateur Keycloak
- Un utilisateur = une identité unique Keycloak

4.2 Attributs utilisateur

Exemples d'attributs stockés dans Keycloak : - userId (UUID) - email - prénom / nom - provider (google | facebook | local) - date de création

5. Autorisation (Access Control)

5.1 Modèle d'autorisation

Approche **RBAC enrichie par des privilèges** : - Un **rôle** est un regroupement logique de **privilèges** - Les **privilèges** représentent les permissions atomiques réelles - Les utilisateurs se voient attribuer des rôles, jamais des privilèges directement

Cette approche permet : - Une granularité fine - Une meilleure évolutivité - Une gouvernance claire des accès

5.2 Rôles

Exemples de rôles : - USER - PREMIUM_USER - ADMIN

Chaque rôle agrège un ensemble de privilèges.

5.3 Privilèges

Exemples de privilèges : - READ_TRANSACTIONS - CREATE_TRANSACTION - UPDATE_TRANSACTION - DELETE_TRANSACTION - EXPORT_DATA - MANAGE_USERS

Mapping exemple : - USER → READ_TRANSACTIONS, CREATE_TRANSACTION - PREMIUM_USER → READ, CREATE, EXPORT_DATA - ADMIN → tous les privilèges

Les privilèges sont : - Gérés dans Keycloak (via rôles composites ou attributes) - Injectés dans le JWT - Exploités par Spring Security

6. Sécurisation du Backend (Spring Boot)

6.1 Configuration

- Spring Security
- OAuth2 Resource Server
- Validation JWT (issuer, audience, signature)

6.2 Exploitation des rôles et privilèges

- Les rôles sont utilisés pour le **contrôle d'accès global**
- Les privilèges sont utilisés pour le **contrôle fin des endpoints**

Exemples : - Accès API lecture : privilege READ_TRANSACTIONS - Export de données : privilege EXPORT_DATA - Administration : privilege MANAGE_USERS

Implémentation recommandée : - Mapping JWT → GrantedAuthorities - Préfixes : ROLE_ et PRIV_

7. Sécurisation du Frontend (Angular)

- Intégration OIDC avec Keycloak
 - Redirection systématique vers Keycloak pour authentification
 - Guards Angular basés sur rôles et privilèges
 - Stockage du token en mémoire uniquement
 - Rafraîchissement automatique des tokens
-

8. Authentification forte – MFA (OTP)

8.1 Principe général

L'authentification **à deux facteurs (MFA)** est **obligatoire** pour tous les utilisateurs.

Après une authentification réussie (Google, Facebook ou locale), l'utilisateur doit fournir un **OTP** généré par une application d'authentification.

8.2 OTP

- Basé sur l'algorithme **TOTP (RFC 6238)**
- Généré par des applications compatibles :

- Google Authenticator
 - Microsoft Authenticator
 - Authy
-

8.3 Gestion dans Keycloak

- MFA activé au niveau du Realm
 - OTP obligatoire via une **Authentication Flow personnalisée**
 - Enrôlement OTP imposé lors de la première connexion
 - Refus d'accès tant que l'OTP n'est pas configuré
-

8.4 Expérience utilisateur

1. Login via Google / Facebook / Local
 2. Vérification identité réussie
 3. Demande OTP
 4. Accès autorisé uniquement après validation OTP
-

9. Sécurité des données

8.1 Données sensibles

- Les mots de passe ne transitent jamais par le backend
- Les données financières sont stockées en base PostgreSQL

8.2 Bonnes pratiques

- HTTPS obligatoire
 - Chiffrement des communications (TLS)
 - Logs sans données sensibles
-

9. Traçabilité et audit

- Journalisation des événements clés :
 - Connexion
 - Déconnexion
 - Échecs d'authentification
 - Exploitation des événements Keycloak
 - Possibilité d'export vers un SIEM
-

10. Évolutivité et bonnes pratiques

- Ajout futur d'autres providers (Apple, LinkedIn)
 - Activation possible de MFA (2FA)
 - Séparation des environnements (DEV / REC / PROD)
 - Rotation des clés JWT
-

11. Conclusion

Cette architecture de sécurité repose sur un **Identity Provider robuste**, respecte les standards du marché et offre une expérience utilisateur fluide tout en garantissant un haut niveau de sécurité.

Elle est parfaitement adaptée à une application financière grand public.

Auteur : Yassin MELLOUKI

Technologies : Keycloak, OAuth2, OIDC, Spring Security, Angular

Date : 2026-02-07